



Hewlett Packard
Enterprise

UNIFIED MONITORING HEALTH REPORTING AND INSIGHTS OF MONITORING COMPONENTS



JSS Science and Technology University

CTY Members:

Aneesh Aveen Sunganahalli, Mithun S, Navan, Vaibhav M N, Vinyas Bharadwaj

CTY Mentor - Raghul Vasudevan

07-07-2026



CONFIDENTIAL | AUTHORIZED

Project Name : Unified Monitoring Health Reporting and Capacity Planning Advisor

College / University Name : JSS University of Science and Technology

Project Participants: Aneesh Aveen Sunganahalli, Mithun S, Navan, Vaibhav M N, Vinyas Bharadwaj

Mentor Names : Raghul Vasudevan, Prarthana G

Project Code/ GitHub Link: <https://github.com/aneeshsunganahalli/HPE-Monitor>



Problem Statement

HPC and large-scale system administrators rely on critical monitoring stacks—such as Apache Kafka, Logstash, OpenSearch, Prometheus, and Grafana—to maintain operations. However, these infrastructure components are inherently resource-intensive:

- They **consume** significant **disk** space, **memory**, **CPU**.
- **Resource demands fluctuate** aggressively based on system scale, erratic workload volatility, and sudden telemetry bursts.
- **Administrators currently lack** unified, real-time visibility into the health of these distributed components, making it difficult to detect anomalies or predict future storage and compute breaches.

Without deep observability and predictive insights, administrators are forced into reactive resource allocation:

- **Under-provisioning:** Risks pipeline lag, data loss, and system crashes during ingestion spikes.
- **Over-provisioning:** Incurs unnecessary cost overheads by reserving idle compute and storage.

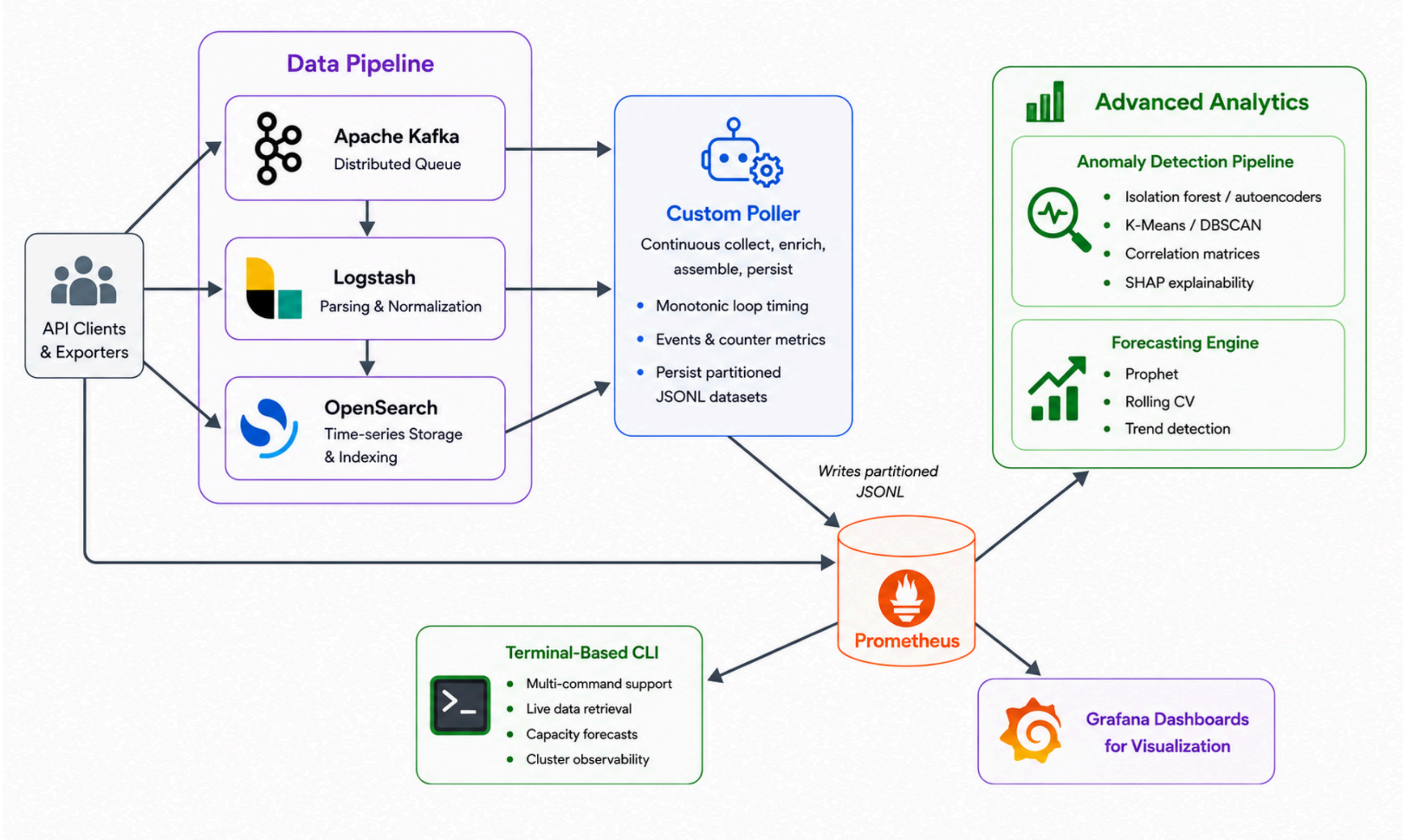
Proposed Solution: Data-Driven Observability & Planning

The Unified Monitoring Health Reporting and Capacity Planning Advisor eliminates guesswork by providing a single, intelligent framework:

- **Real-Time Health Reporting:** A centralized, **terminal-based CLI** for live data retrieval and cluster observability across the entire monitoring pipeline.
- **Proactive Anomaly Detection:** **ML-driven identification** of resource pressures and failure patterns before they escalate.
- **Predictive Capacity Planning:** **Prophet-powered forecasting** that tracks ingestion trends and projects future capacity needs, accounting for daily workload cycles and maintenance noise

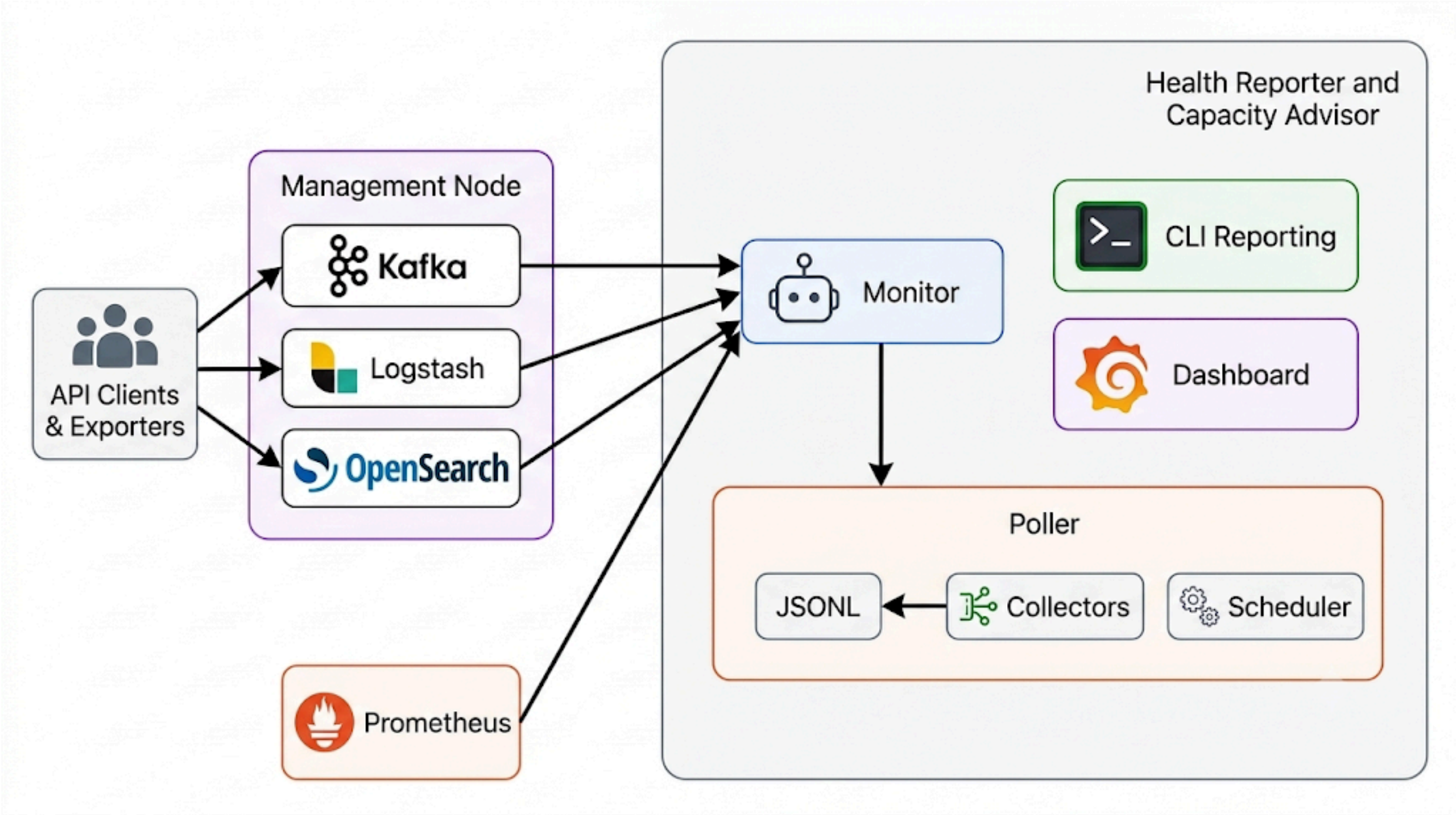


Project Architecture



Pipeline & CLI Architecture

Architecture Diagram



Objectives

- Generating a report of resource utilization of different monitoring stacks.
- Design a modular CLI architecture (entry, routing, views).
- Enable real-time interaction and visualization through CLI.

Skill Set

- Linux / RHEL Administration
- Apache Kafka
- Logstash
- OpenSearch
- System Monitoring Tools
- Basic Networking & Distributed Systems Concepts
- Scripting & Data Handling



System Setup

1. RHEL Installation

- Installed RHEL on local/virtual machine to simulate HPC node
- Configured:
 - Disk partitioning (root, swap)
 - Network settings (static IP / hostname)
- Enabled essential repositories & packages

2. System Configuration

- Installed core utilities:
 - htop, net-tools, sysstat, curl
- Configured:
 - SSH access for remote monitoring
 - User permissions & security policies
- Tuned system for performance:
 - CPU & memory usage optimization

3. Compute Node Exploration

- Monitored system metrics using:
 - top / htop → CPU & process usage
 - free → Memory utilization
 - vmstat → System performance stats
 - sar → Historical metrics

4. Integration Strategy

- RHEL node acts as:
 - Log source for Kafka
 - Metric generator for monitoring pipeline
- Ensures:
 - Realistic data flow
 - Production-like testing environment

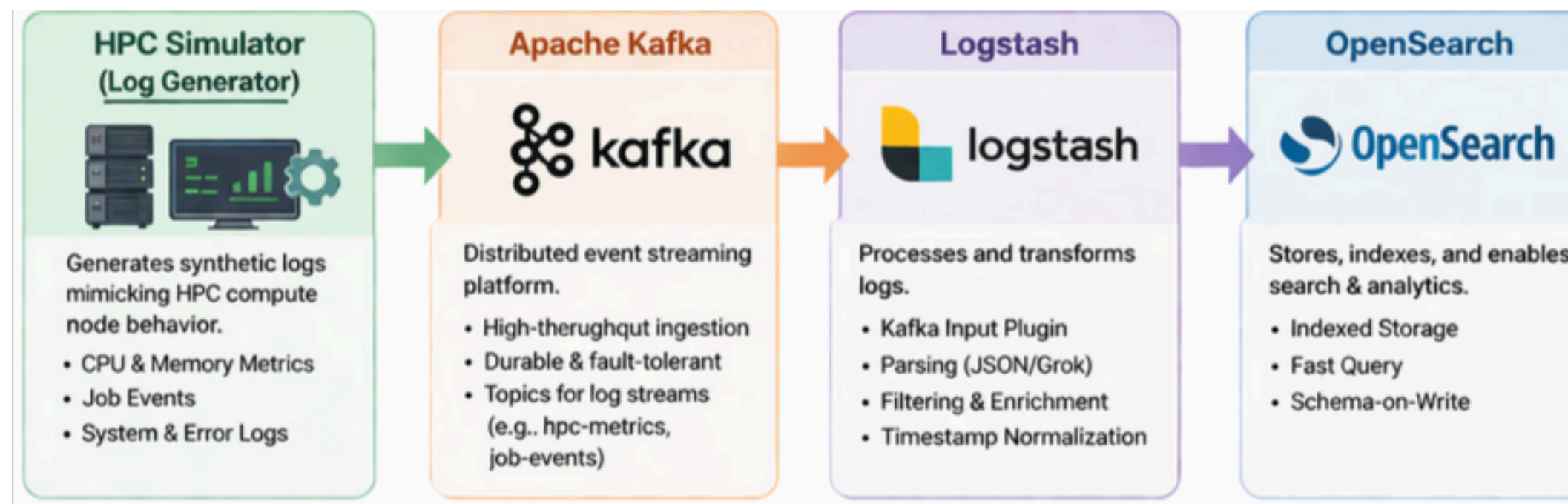
Pipeline Setup

Monitoring stack setup with data pipeline

Overview:

- Log Generator (HPC Simulation)
 - Synthetic logs generated to mimic HPC compute node behavior (CPU, memory, job scheduling, errors)
- Apache Kafka
 - Acts as a distributed message broker
 - Handles high-throughput ingestion of compute node logs
 - Topics created for structured log streams (e.g., hpc-metrics, job-events)

- Logstash
 - Consumes logs from Kafka topics
 - Performs:
 - Parsing (JSON)
 - Data transformation
 - Filtering and enrichment
- OpenSearch
 - Stores processed logs in indexed format
 - Enables fast querying and real-time analytics



CLI Monitor — Overview

Entry Point

`python -m monitor`

Two-Tier Menu Architecture

Service Selector → OpenSearch
Kafka
Prometheus

OpenSearch Sub-Menu → 6 monitoring views, arrow key navigation

Tech Stack

Python — Core language
Click — CLI flag parsing & validation
Rich — Terminal panels, coloured output
simple-term-menu — Arrow-key interactive menus
opensearch-py — OpenSearch REST API client
Prometheus — Historical metric fallback (PromQL)

Usage Examples

Interactive menus (default)

`python -m monitor`

Jump directly to OpenSearch

`python -m monitor --service opensearch`

One-shot Quick Summary

`python -m monitor --summary`

Auto-refresh every 10 seconds

`python -m monitor --summary --watch 10`

Historical trend window (6 hours)

`python -m monitor --timeframe 6h`

Force poller as data source

`python -m monitor --source poller`

CLI Flags & Commands

Flag	Values	Default	Description
--timeframe	30m, 1h, 6h, 4d	1h	Routes metric fetch to live API or historical store based on time window
--source	auto, poller, prometheus	auto	Selects historical trend backend: poller JSONL or Prometheus API
--service	Opensearch, Kafka, Prometheus	—	Skip service selector menu; go directly to named service
--summary	(flag)	off	Skip menus — jump straight to Quick Summary view
--watch	<seconds>	off	Enable auto-refresh mode; re-renders selected view every N seconds



OpenSearch — Monitoring Views

View	What It Shows
1. Quick Summary	5-minute cluster health check: status badge, CPU %, JVM Heap %, RAM, Disk, index activity, shard counts, per-node warnings
2. Historical Trends	Shows trends for CPU, JVM Heap & Indexing Rate; reads poller JSONL first, falls back to Prometheus PromQL API
3. Cluster Health	Detailed cluster status with node count, shard state breakdown, pending tasks, and simple condition explanations
4. Node Performance	Per-node CPU, JVM heap %, RAM, disk + indexing/search counters;
5. Index Deep Dive	All indices sorted by store size (desc); select any index to drill into per-shard layout, states, and node assignments
6. Shard Overview	All shards grouped by state: STARTED / RELOCATING / INITIALIZING / UNASSIGNED; unassigned shards highlighted in red



Prometheus — Monitoring Views

View	What It Shows
1. Snapshot	Prometheus metric health check grouped by process, system, TSDB, and query metrics
2. Query Performance	Shows 99th percentile query latency and active query counts with diagnostics for engine overloads
3. TSDB Deep Dive	Detailed TSDB stats including head series, chunk counts, ingestion rates, and storage size
4. Resource Gauges	Large side-by-side gauges for Prometheus-specific CPU, memory, and disk usage



Kafka — Monitoring Views

View	What It Shows
1. Snapshot	Metric status check covering consumer lag, ISR health, broker count, and CPU/memory/disk resources
2. Broker Performance	Per-broker CPU, memory, and disk usage with plain-English diagnostics for resource pressure
3. Topic Deep Dive	All topics sorted by offset depth; select any topic to drill into per-partition layouts, consumer lag graphs, and retention policies
4. Partition Overview	All partitions grouped by topic showing leader state, ISR counts, replication gap, and consumer lag ratio
5. Resource Gauges	Large side-by-side gauges for CPU, memory, and disk usage with threshold markers"
6. Log Browser	Live viewer for Kafka broker log files with severity color-coding, level filters, and keyword search



Poller Overview

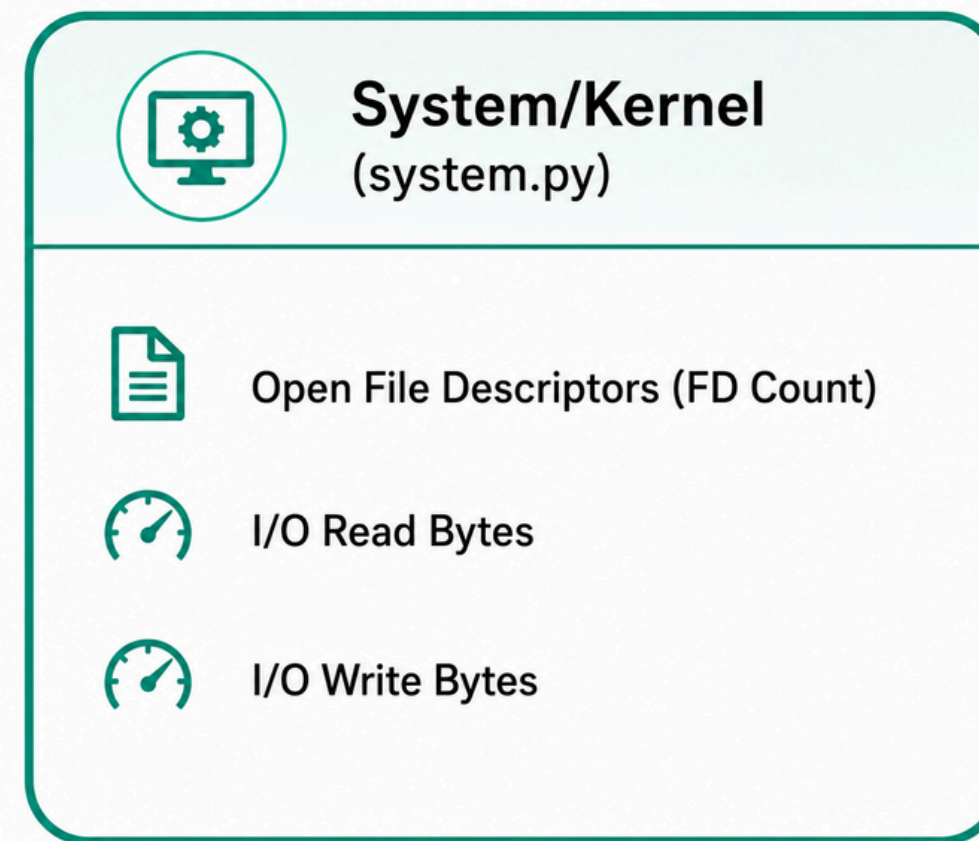
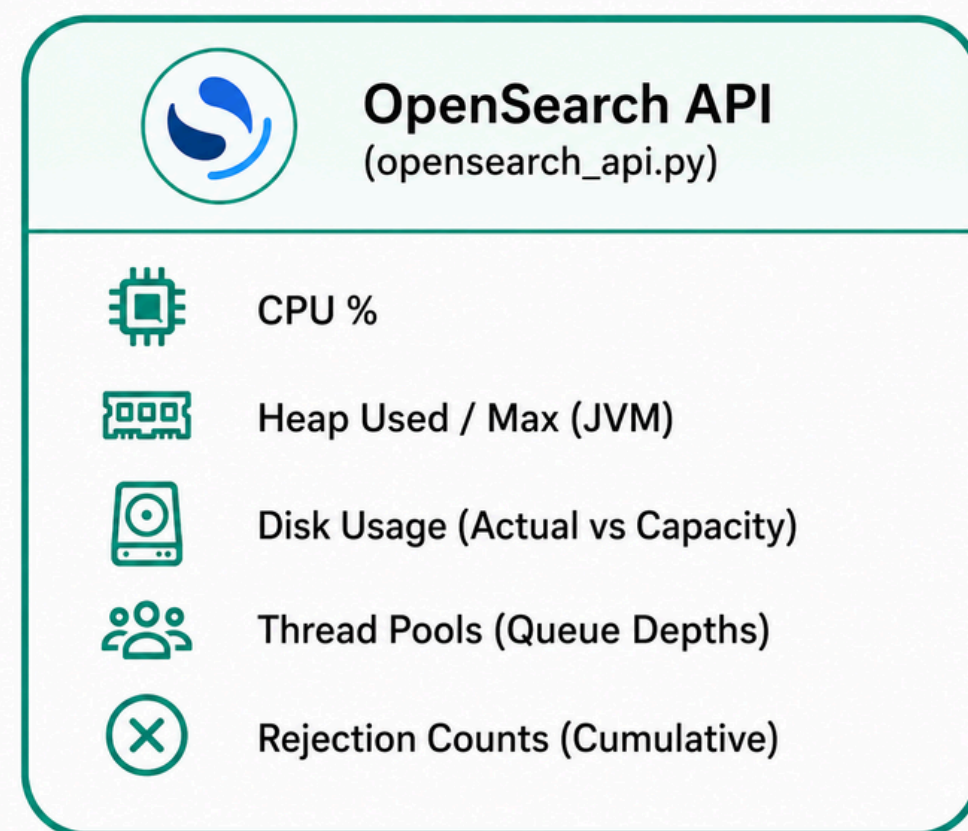
1. Core Architecture

The poller operates as a continuous loop within poller.py. Each iteration (or "tick") of the loop follows a standard sequence:



2. Metric Collection

The poller gathers data from two primary sources:



3. Data Enrichment (Rate Computation)

- **Metric Normalization:** Converts cumulative counters into per-second rates using previous tick data.
- **Performance Tracking:** Computes GC pause and thread rejection rates.
- **Throughput Conversion:** Translates cumulative I/O bytes into bytes-per-second (BPS).

4. Persistence (writer.py) :

- **JSONL Format:** Appends records as single JSON lines.
- **Automatic Rotation:** Partitions logs by date.
- **High Efficiency:** Uses low-overhead, high-frequency appending.
- **Configurable Location:** Defaults to /var/log/hpe-monitor/poller

5. Execution & Configuration

- **CLI Execution:** `python -m poller --interval 30 --verbose`.
- **Adjustable Interval:** Defaults to 30s in config.py.
- **Stable Cadence:** Adjusts sleep time for consistent polling.



3. Data Enrichment (Rate Computation)

- **Metric Normalization:** Converts cumulative counters into per-second rates using previous tick data.
- **Performance Tracking:** Computes GC pause and thread rejection rates.
- **Throughput Conversion:** Translates cumulative I/O bytes into bytes-per-second (BPS).

4. Persistence (writer.py) :

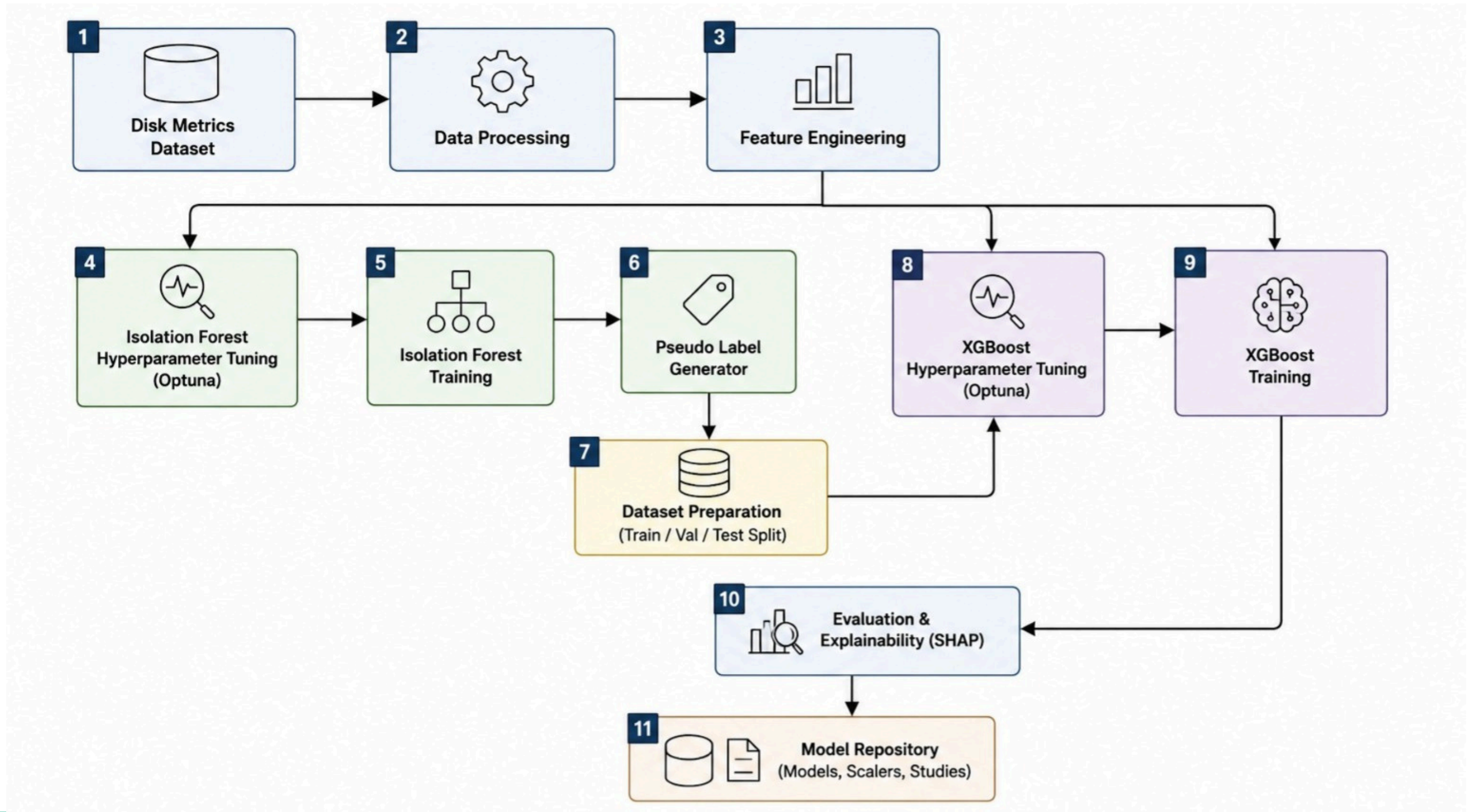
- **JSONL Format:** Appends records as single JSON lines.
- **Automatic Rotation:** Partitions logs by date.
- **High Efficiency:** Uses low-overhead, high-frequency appending.
- **Configurable Location:** Defaults to /var/log/hpe-monitor/poller

5. Execution & Configuration

- **CLI Execution:** `python -m poller --interval 30 --verbose`.
- **Adjustable Interval:** Defaults to 30s in config.py.
- **Stable Cadence:** Adjusts sleep time for consistent polling.



Anomaly Detection Architecture



Anomaly Detection Overview

Unsupervised Baselines

Detects outliers via Isolation Forest without historical labels.

Automated Tuning

Uses Optuna to set a strict 2% anomaly threshold.

Data Balancing

Injects Gaussian noise for a 50/50 training data split.

Supervised Prediction

Trains XGBoost on pseudo-labels to predict failure patterns.

Leakage Prevention

Drops Isolation Forest scores to reveal true physical root causes.

Transparent Explainability

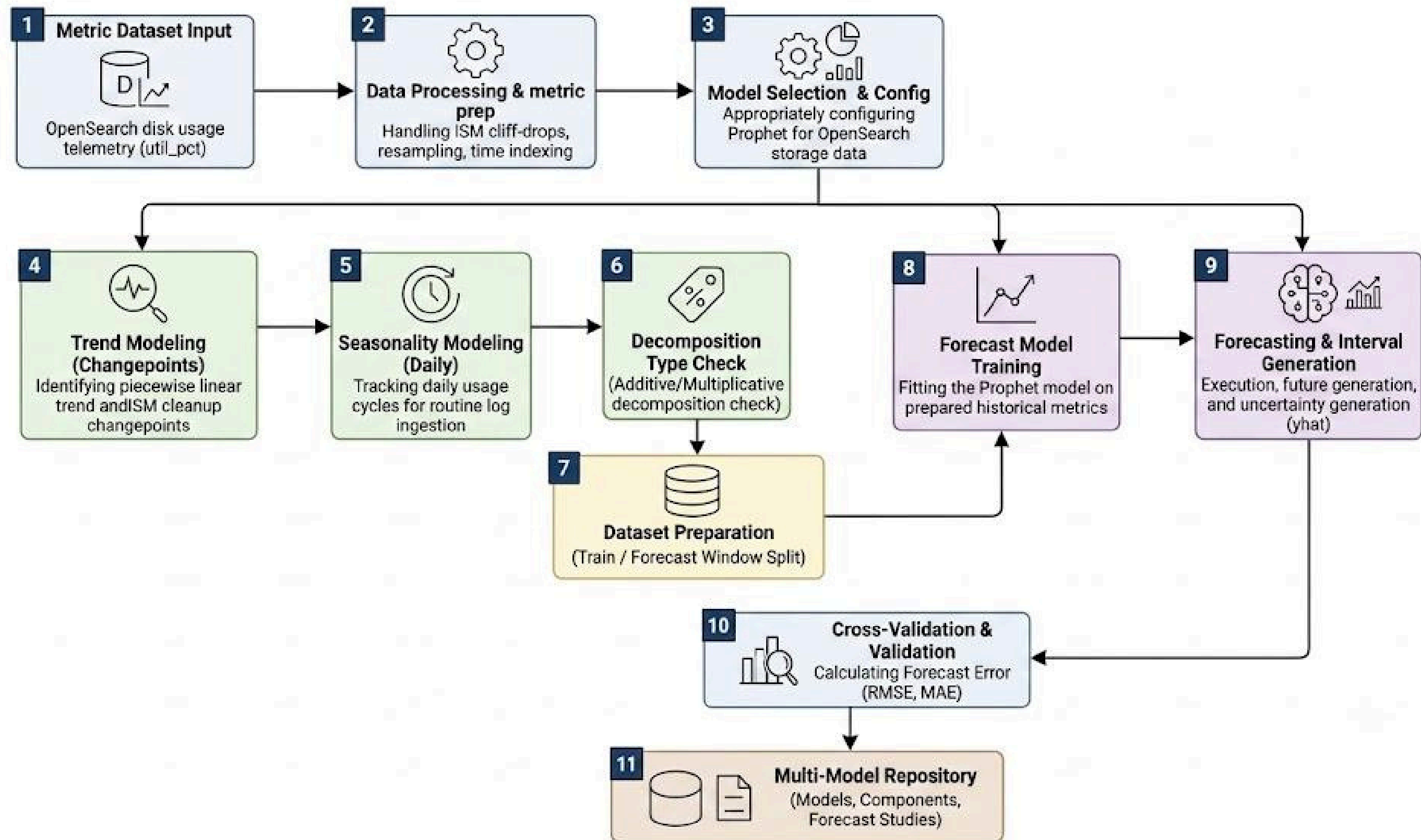
Uses SHAP to identify the exact metrics triggering alerts.

Centralized Repository

Stores trained models, scalers, and tuning studies.



Forecasting Architecture



Forecasting Overview

Prophet is utilized to analyze disk usage data by treating it as a continuous, flexible curve.

Native Seasonality

Adjusts for predictable daily workload cycles.

Step-Change Adaptability

Handles abrupt log rotations without model failure.

Resampled Smoothing

Uses 10-minute intervals to reduce metric noise.

Optimized Parameters

Leverages Grid Search to find the best configuration.

Cross-Validation

Validates accuracy using rolling historical windows.

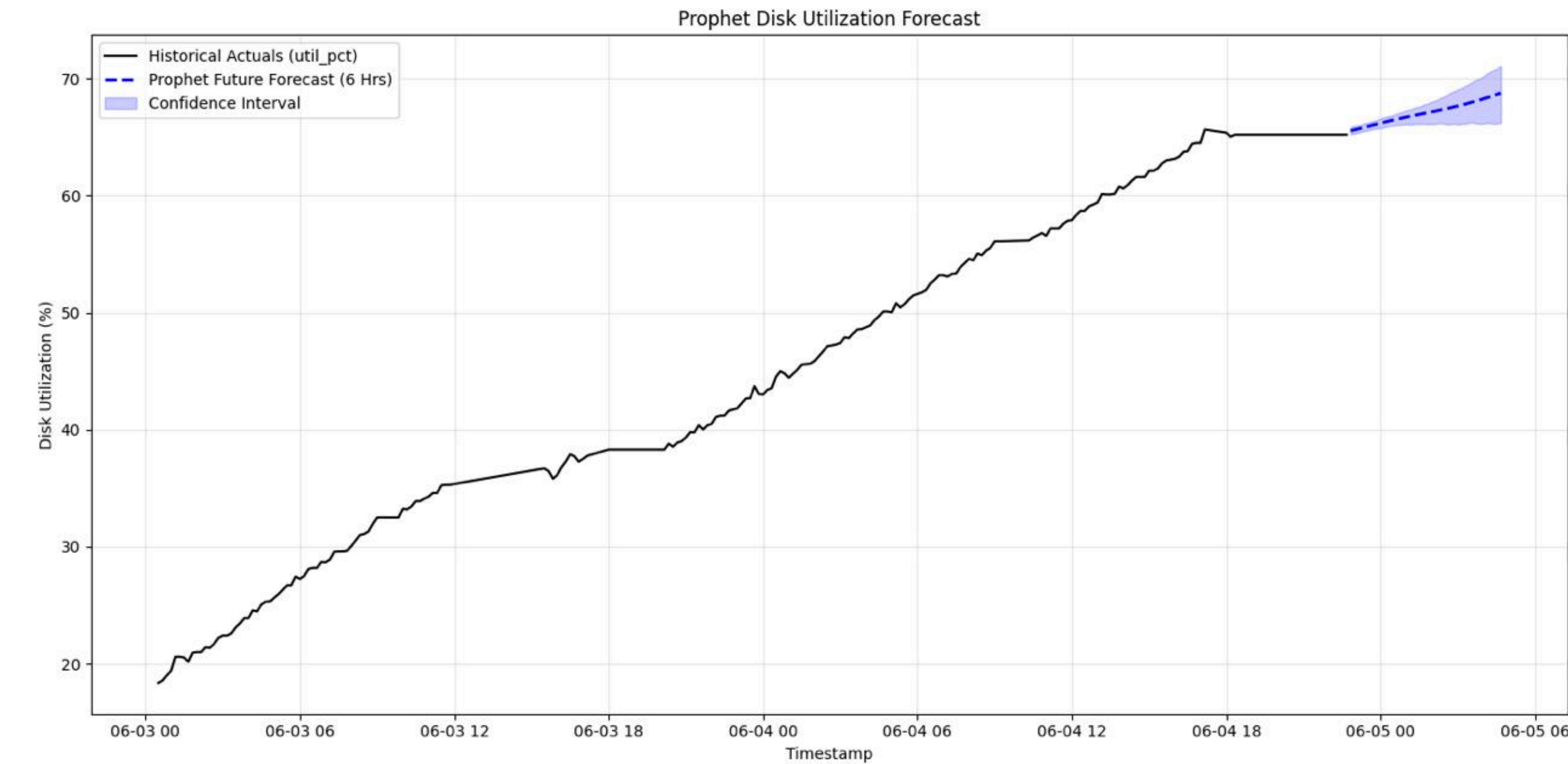
Confidence Intervals

Projects upper and lower uncertainty bounds.

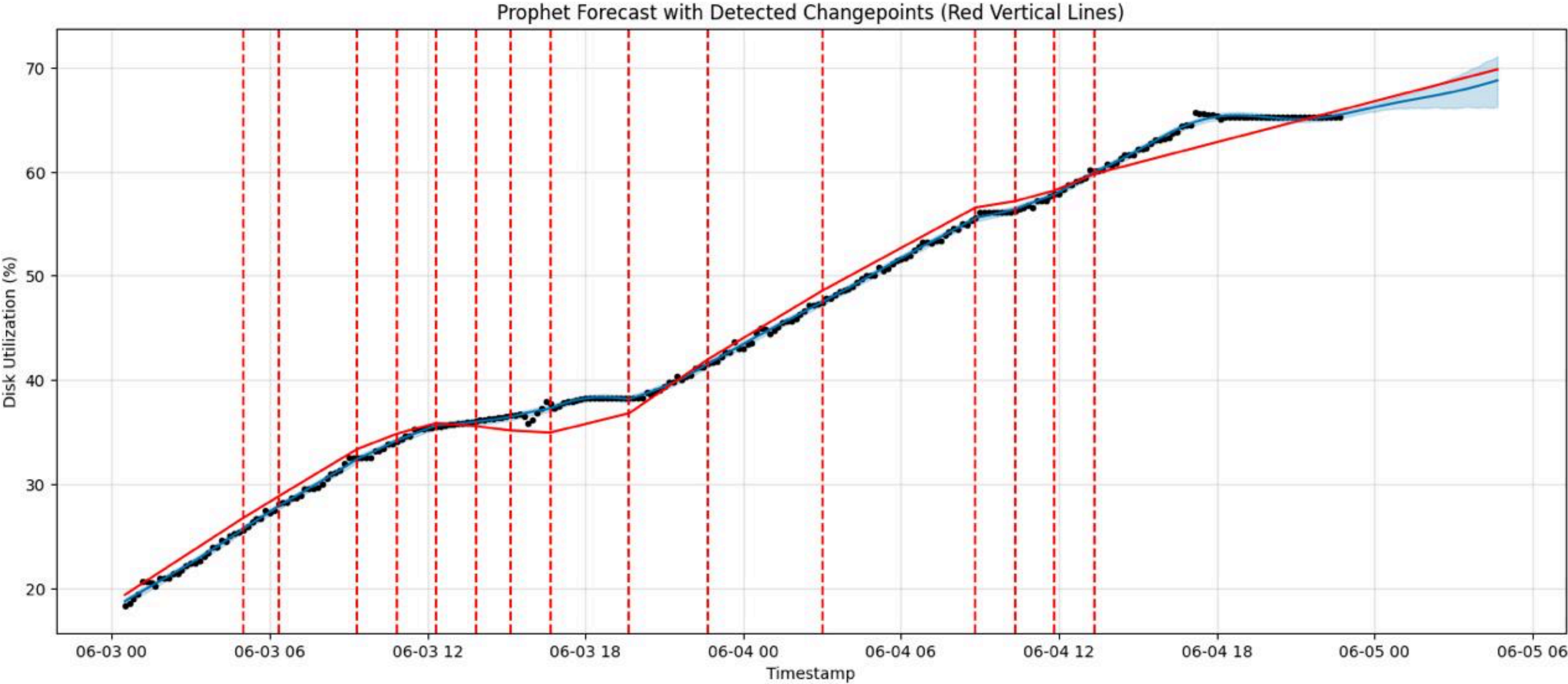
Trend Tracking

Detects change-points to follow actual ingestion rates.

Forecasting Overview



Forecasting Overview



Challenges

Pipeline & CLI

Integration Complexity

Connecting Logstash, Kafka, and OpenSearch securely across distributed networks.

Anomaly Detection

Label Leakage

Second Model cheated by relying on the first model's guesses instead of looking at the actual data.

Severe Class Imbalance

Rarity of genuine anomaly events compared to normal operational data.

Forecasting

Workload Volatility

Erratic ingestion spikes mask long-term growth baselines.

Insufficient History

Limited data makes projecting distant capacity breaches difficult.



Key Learnings

Pipeline & CLI

- **Reliable Flow:** Kafka buffers logs to prevent data loss during spikes or crashes.
- **System Monitoring:** Prometheus and Grafana track data pipeline health and performance.
- **Data Processing:** Logstash cleans and structures raw logs for OpenSearch analysis.

Anomaly Detection

- **Leakage Prevention:** Learns physical root causes, not just detector mimicry.
- **Data Balancing:** Uses Gaussian Noise instead of extensive labels.
- **Explainability:** SHAP identifies key driving metric.

Forecasting

- **Resilience:** Models continuous curves, ignoring noisy maintenance events.
- **Accuracy:** Captures daily seasonality for precise ingestion peaks.
- **Proactive:** Replaces threshold alerts with future projections.



Next Steps

- **Local Anomaly Alerting:** Integrates lightweight machine learning (like an Isolation Forest) to instantly flag unusual resource spikes against historical baselines before they trigger hard alerts.
- **Predictive "Time-to-Exhaustion" Metrics:** Analyzes storage and memory consumption trends to give administrators an exact countdown (e.g., "4.5 days remaining") until a volume hits capacity.
- **Simulated Workload Forecasting:** Adds a interactive command (like `--scale-workload 2x`) to model future resource usage, making terminal-based capacity planning effortless.



THANK YOU

